

Faster than Light: Latency Measurement Errors in Message-Broker Benchmarks

Gustavo Pedro Ricou and Romaric Duvignau

Abstract—Message-broker benchmarks now compare systems on sub-millisecond paths, where the time being measured is shorter than the clock’s resolution and than a busy thread’s wait for a processor. There a benchmark can misreport in two ways its output does not announce. First, a latency timed from the broker’s acknowledgment turns negative whenever the thread recording it is scheduled late by more than the delivery. Timed on one clock, 62,264 of our 738,730 messages were negative this way and none from the send, so the cause is not clock synchronization; real-time priority for the timestamping threads, at unchanged utilization, cut the rate 7–80×. A zero-cost sign check rejected 1,321 of our 2,266 runs, including every run behind our first result. Second, the OpenMessaging Benchmark timestamps in whole milliseconds and silently discards every latency that is not positive: 71 of 75 embedded-mode settings whose summary we captured printed a median of exactly 1.0 or 2.0 ms, computed from as little as 0.36% of the samples. We derive the retained fraction from the ratio of delivery to resolution, confirm a registered prediction of it, find uncounted disposal in five of ten tools, and give benchmark authors checks that cost nothing.

Index Terms—benchmarking; clock synchronization; latency measurement; measurement errors; message brokers; scheduling; stream processing; timestamp resolution.

I. INTRODUCTION

OUR first harness timed each message from the moment the producer learned that the broker had accepted it. By its arithmetic, 62,264 of the 738,730 messages we timed arrived before they were sent. By the clock, none did: timed from the send on the same clock, not one of them is negative (Fig. 1a). The arithmetic had subtracted two timestamps written by two different threads, and a thread must wait for a processor before it can read the clock. That wait is the *scheduling delay*: the time a thread that is ready to run spends queued before a core runs it. When the thread that records the acknowledgment waits longer than the message takes to arrive, the difference comes out negative. Nothing arrived early; one reading was taken late. Users of the benchmark discussed next have met the same ordering, a publish latency above the end-to-end latency on a single node. They were told it was clock skew between nodes [1].

That benchmark, the OpenMessaging Benchmark [2], is the one broker comparisons share. It has a sensible rule for such faster-than-light samples, which it expects from machines whose clocks disagree: it discards every latency that is not

positive [3]. But its end-to-end clock counts whole milliseconds, and to that clock a message that arrives before the digit changes took no time at all, which is faster than light too. The same rule therefore discards every such message: on the benchmark’s own path about half of them, and at some send rates nearly all (Section IV-C). Applied to our own corpus, it would delete 45.8% of the spans without a count (Fig. 1b). In 71 of the 75 settings whose own summary we captured, it printed a median of exactly 1.0 or 2.0 ms. The median came from as little as 0.36% of the samples taken, and nothing in the output said how few.

Neither failure announces itself. The second leaves a signature that nobody reads: every end-to-end percentile the benchmark prints sits on the millisecond grid. It also makes the benchmark look better. A median computed from 0.36% of the samples is typographically identical to one computed from all of them, and more stable from run to run for having its variation deleted. The tool passes the reproducibility a reader checks because it fails the verifiability nobody does. Nor is this one project’s defect. The tools that time broker paths are few and shared, and Apache Kafka’s own end-to-end tool reports zero at every percentile on such a path [4]. Clock resolution and scheduling delay are each well understood. What was not known is what they do to the sub-millisecond comparisons that broker benchmarks now publish, because the tools that produce those comparisons discard the evidence that would show it.

A. Contributions

We make three contributions, each established by manipulation rather than by observation alone.

- 1) **The first failure, and its cause** (Section III). A latency referenced to an acknowledgment timestamp turns negative when the thread that writes it is scheduled late by more than the delivery. Real-time priority for the timestamping threads cut the rate 7–80× at unchanged utilization. Two placements of the same load at identical utilization differed twofold, so utilization alone does not set the rate. The same mechanism appears in Kafka’s own Java client and on a second processor family.
- 2) **The second failure, derived and confirmed** (Section IV). With a millisecond clock and a positivity filter, the share of samples a benchmark keeps is set by the ratio of the delivery to the clock’s resolution and by the phase of its send schedule. A registered manipulation of message size moved a setting across the boundary this predicts. Five of ten tools we read at source dispose

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

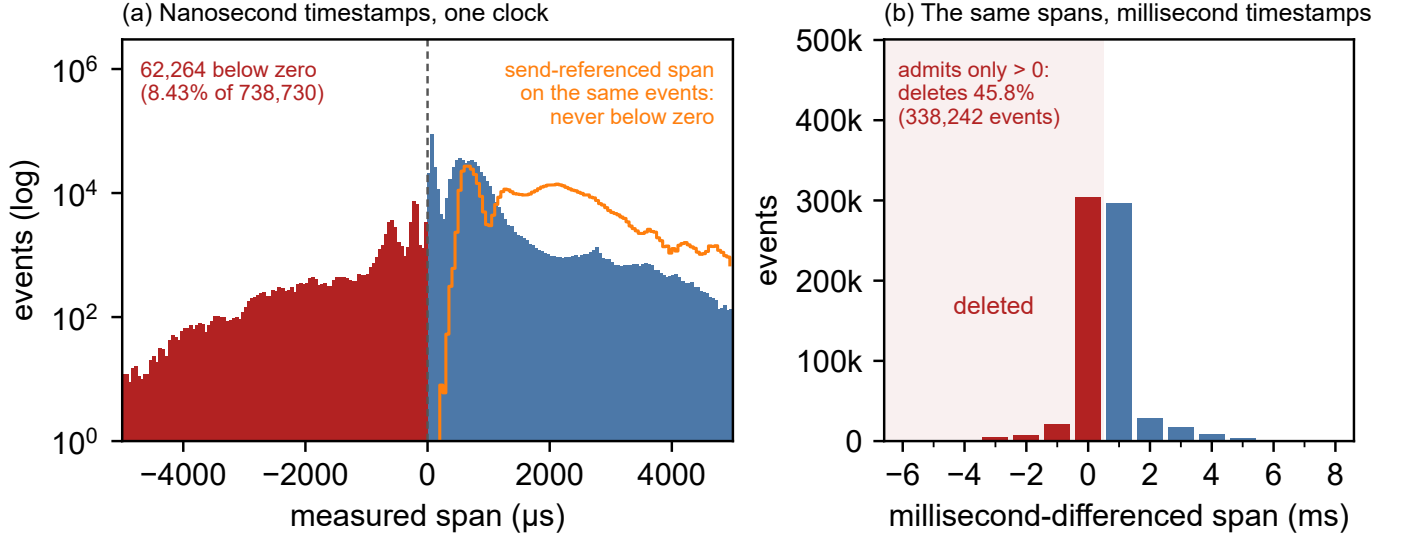


Fig. 1. **Both failures on one population.** The 738,730 messages of our corpus, timed on one clock. (a) Timed from the broker’s acknowledgment, 62,264 of them come out negative (red); timed from the send, none does. (b) The same spans as a millisecond timestamp would record them: a span shorter than a millisecond computes to 0 or 1 ms, and a filter that keeps only positive values, as the benchmark of Section IV does, would delete 45.8% of them without a count.

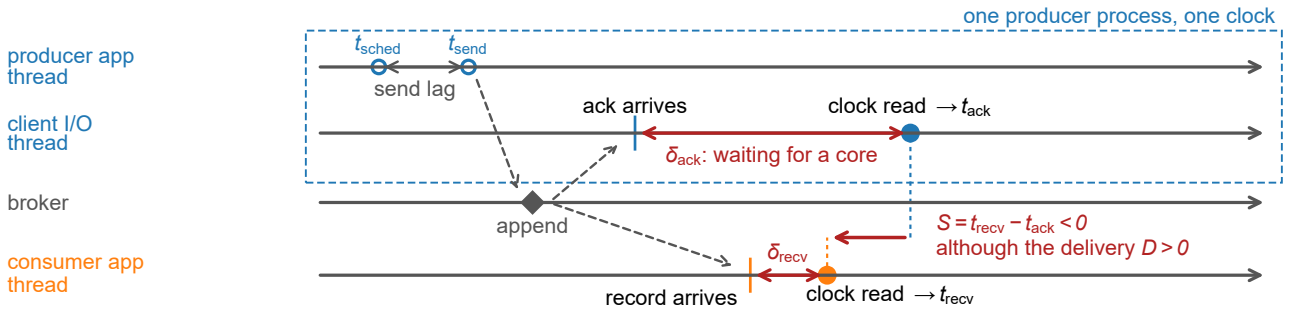


Fig. 2. **Late timestamp, not an early record.** How a positive latency is measured as negative, on one clock. The producer’s application thread takes t_{sched} and t_{send} . Its client I/O thread takes t_{ack} , but only after waiting δ_{ack} for a core. When that wait outlasts the delivery, the consumer’s t_{recv} precedes t_{ack} , and S is negative although D is positive. The path drawn is Kafka’s; Redis timestamps the acknowledgment on the application thread and inverts anyway, because the wait for a core is charged either way.

of such samples without counting them, and a registered audit of 43 published reports found the signature in eight configurations and a stated retention in none (Section V).

- 3) **Checks that cost nothing** (Section VI). A per-run sign check rejected 1,321 of our 2,266 runs, every run behind our own first result among them. All had passed every conventional check we knew. Publishing the retained fraction beside every latency exposes the second failure the same way.

Both processes read one clock unless the text says otherwise, so nothing here depends on clock synchronization.

II. HOW A BENCHMARK TIMES A MESSAGE

A producer process sends recorded events at a paced rate to a broker, and a consumer process reads them back. Four timestamps bracket a message’s path. The first, t_{sched} , is when the event was due to be sent, and t_{send} is taken immediately before the send call. Then t_{ack} is when the producer learns

that the broker accepted the message, and t_{recv} is when the consumer holds the record. Three quantities follow:

$$\begin{aligned} D &\equiv t_{\text{recv}} - t_{\text{send}}, & A &\equiv t_{\text{ack}} - t_{\text{send}}, \\ S &\equiv t_{\text{recv}} - t_{\text{ack}} = D - A. \end{aligned} \quad (1)$$

D is the *delivery time*, from the send call to the consumer holding the record. A is the *acknowledgment lag*, from the send call to the producer observing the broker’s acknowledgment. S is what a harness reports when it times a message from the acknowledgment rather than from the send, as our own first harness did. We call it the *transport proxy*, for the reason given below. The end-to-end latency runs from the event falling due to the consumer holding it:

$$E = t_{\text{recv}} - t_{\text{sched}} = \underbrace{(t_{\text{send}} - t_{\text{sched}})}_{\text{send lag}} + A + S. \quad (2)$$

The *send lag* is how late the send call ran after the event was due. A *span* is any difference of two of these timestamps, and a *negative span* is one whose difference is below zero.

Write τ for the *timestamp resolution*, one millisecond in the benchmark of Section IV. Write T_{true} for the *true delivery time* of a message, the interval D would report if its timestamps were arbitrarily fine. *Retention* is the fraction of the samples a measurement tool takes that survive into what it reports.

A. The transport proxy is not a causal chain

D and E are chains of events, each preceding the next. S is not. The broker's append precedes both the producer's receipt of the acknowledgment and the consumer's receipt of the record, but neither of them precedes the other. They are two branches of one cause. Since Equation 1 holds per event, whatever the signs,

$$S < 0 \quad \text{if and only if} \quad A > D. \quad (3)$$

The identity is checked on every build: both sides count 62,264 negatives over our 738,730 events, and the build fails if they part. Every timestamp is written some time after the event it marks, because the thread that reads the clock must first be scheduled (Fig. 2). That delay is not the cost of the clock call, which is small and separately measurable [5]. It is the wait before the call runs at all. A negative S therefore does not show that either timestamp is wrong. It shows that the acknowledgment cannot serve as the origin of a delivery latency for that event, which is the use an acknowledgment-referenced harness puts it to. The same holds wherever the origin timestamp is written by a thread that must first be woken, such as a delivery-report poll or a completion reaper (Supplement S30). A fixed distribution of A overlaps a small D almost entirely and a large one hardly at all. That is why a path whose transport runs to tens of seconds passes every run, while the same hardware measuring one millisecond fails wholesale.

B. Testbeds, clocks and workload

The measured findings of Sections III and IV come from four Oracle Cloud VM. Standard.E5.Flex instances, three brokers and one driver. They run Ubuntu 22.04 on kernel 6.8.0-1057-oracle (EEVDF scheduler) and CPython 3.10 on a private network, with *chrony*-disciplined clocks logged every minute. The producer and the consumer run as separate processes on one host and read one clock (`time.time_ns()`). A workstation running the brokers under containers enters only through the audit of Section III-A. One early phase perturbed the measurement enough to be unusable and is excluded from every result here; the campaigns that remain measure utilization with no core pinning. A later registered campaign ran on three pairs of Microsoft Azure machines, two x86 and one Neoverse N2. The workload is StatsBomb's open football event data [6]: eleven of its 3,315 matches, replayed in order at a chosen send rate. It stands for small, frequent, ordered messages, and nothing below depends on its content beyond message size and rate. The achieved send rate is recorded per run against elapsed wall time, and pacer jitter is 66.3–69.2 μs at p90 over 24 runs.

C. Terms

A *condition* is one point of the experimental design: a broker, a send rate, a message size, a background load and a consumer pattern. A *cell* is a condition together with a workload; where the workload is fixed the two coincide. A *run* is one execution of a cell, and a *replicate* is a further run of the same cell. Where a manipulation compares settings that differ in one variable, each setting is a *configuration* of that manipulation. Utilization ρ is the fraction of processor time busy on the host, as measured over the run. The core count k is the number of cores the background load occupies. Two settings that reach the same utilization with the load on different numbers of cores differ in *load geometry*: *concentrated* on few cores or *spread* over many.

D. The sign check

A run is rejected if more than 1% of its events carry a negative value in any latency component, or if the median of any component is negative. A condition is usable only if all of its runs survive. We fixed the rule before the final campaign, after earlier campaigns had run, and apply it to every run, including those whose results looked reasonable. The one-percent line is a convention, and it decides nothing we report: swept from 0 to 20%, it makes none of our first result's rejected conditions usable (Supplement S18). The reason is not that a negative is impossible, since for the transport proxy it is not: a run whose reference timestamp fails on more than one event in a hundred cannot report a latency, whatever the cause. A run that passes has cleared the cheapest available bar and nothing more. Every proportion below is a count over a stated denominator. The rates of Table II carry Wilson score 95% intervals and each of its ratios a Katz interval, Holm-corrected within a named family; the corrected value is the one the supplement's tables display. Every table is generated from the committed data at build time.

III. FAILURE 1: LATE TIMESTAMPS INVERT THE ACKNOWLEDGMENT-REFERENCED SPAN

The negatives in our corpus are produced by scheduling. We establish this by moving scheduling while holding everything else fixed.

A. What the sign check finds

The check rejects 862 of 1,382 runs on the workstation testbed (62.4%) and 459 of 884 on the cloud testbed (51.9%). Every run behind our first result is among them: on that first testbed 8 of 76 conditions survive. Every rejected run had passed every conventional check we knew. Our first corpus had survived six corrections, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction. It was still entirely invalid, and nothing in its output said so (Supplement S1.14).

Table I separates the spans over a wider population: every cloud run that produced matched events, 5,913 of them, since a condemned run answers which span inverts as well as a clean one does. The acknowledgment-referenced span is negative on

TABLE I

NEGATIVES BY SPAN AND BROKER (5,913 RUNS, 738,730 EVENTS, ONE CLOCK). ONLY THE ACKNOWLEDGMENT-ORIGIN SPAN INVERTS, AND IT DOES SO UNDER BOTH BROKERS: KAFKA ON 8.67% OF ITS 367,894 EVENTS, REDIS ON 8.19% OF ITS 370,836, AND THE CAUSAL CHAINS ARE CLEAN UNDER BOTH — THE ACKNOWLEDGMENT LAG A AMONG THEM, ITS SMALLEST RUN MINIMUM 112 μ s CLEAR OF ZERO.

Span	Origin timestamp	Kafka	Redis	All
$t_{\text{recv}} - t_{\text{ack}}$	acknowledgment	31,899	30,365	62,264
$t_{\text{recv}} - t_{\text{send}}$	send (chain)	0	0	0
$t_{\text{out}} - t_{\text{send}}$	send (chain)	0	0	0
$t_{\text{ack}} - t_{\text{send}}$	send (chain)	0	0	0
E , end-to-end	emission	0	0	0

62,264 of 738,730 events (8.43%), and 3,976 runs exceed the threshold on it. The send-referenced span, the acknowledgment lag A and the end-to-end latency E of Equation 2 are negative on no event at all.

The sign sees only the worst cases. The acknowledgment lag exceeds half the delivery time on 74.5% of events and a tenth of it on 95.2%, but the check fires only on the 8.4% where it exceeds the whole. The two delays are correlated within a run (median correlation 0.80 over 70 conditions), one scheduler causing both. Treating the two as independent within a run would overpredict them 7.4-fold, the median of the 54 conditions above 0.1%. A delivery longer than the acknowledgment lag is therefore not clean, only quiet. Load-induced inflation that stays positive passes the check by construction: the check is necessary, not sufficient.

B. The cause, by manipulation

Each timestamping thread is either running, in which case its timestamp is late by a narrow jitter, or preempted and off the processor. Write p for the probability of the second state, the thread's *occupancy*. To leading order the negative-span rate is the occupancy times the chance that a residual stall outlasts the delivery. Load changes how often the thread that records the time is off the processor, not how wide the ordinary jitter is. From idle to the knee the rate grows 61 $\times$ and the jitter's width only 5 $\times$; the highest rate near saturation, 0.37, is a maximum over those conditions and not a bound (Supplement S9). Load moves p but does not fix it, as the geometry rows below show, so we write p and not $p(\rho)$. The model predicts how the rate falls as the path lengthens, not how it rises with load.

Real-time priority on the timestamping threads moves the occupancy while utilization stays fixed. The rate falls by factors of 39 and 54 (Table II, top). Across 8 matched pairs from 60 to 95% utilization the factor ranges from 7 to 80, every pair with disjoint intervals (Supplement S12). The residual rates settle near the floor the model names in advance, the rate when the thread is never preempted: $C_0 \approx 0.004$. Over all 8 pairs they run from 0.0017 to 0.0144, so C_0 is where the rate settles and not a bound it respects. Utilization did not change, and the rate changed fiftyfold. That refutes any account in which utilization alone sets the rate, including the queueing form we had ourselves adopted (Supplement S7).

At $k = 6$ loaded cores, two placements of the same load reach $\rho = 0.7531$, identical to four decimals. Their negative-

TABLE II

MECHANISM, BY MANIPULATION (2,985 MATCHED EVENTS PER CELL). TOP: TIMESTAMPING THREADS MOVED TO SCHED_FIFO WITH THE BACKGROUND LOAD UNCHANGED; THE ACHIEVED UTILIZATION ρ OF THE TWO CONFIGURATIONS AGREES TO WITHIN 0.0006. BOTTOM: ONE UTILIZATION REACHED BY TWO LOAD GEOMETRIES AT $k=6$ LOADED CORES, IN AN ORIGINAL CAMPAIGN AND A FULL REPLICATION.

Manipulation	Configuration	ρ	Rate	95% CI	Factor
Priority, 75%	ordinary	0.7531	0.1320	0.1203–0.1446	39 $\times$
	real-time	0.7525	0.0034	0.0018–0.0062	
Priority, 88%	ordinary	0.8809	0.3049	0.2886–0.3216	54 $\times$
	real-time	0.8812	0.0057	0.0036–0.0091	
Geometry, original	concentrated	0.7531	0.0824	0.0731–0.0928	2.07 $\times$
	spread	0.7531	0.1709	0.1578–0.1848	
Geometry, repl.	concentrated	0.7531	0.0600	0.0520–0.0691	2.05 $\times$
	spread	0.7531	0.1229	0.1117–0.1352	

span rates differ by 2.07 $\times$, reproduced at 2.05 $\times$ by a full replication (Table II, bottom). Padding the payload lengthened transport 77 $\times$ at a 0.012 utilization spread, and the rate fell 4.1 $\times$. That is the direction Equation 3 predicts and the opposite of a stress account. Finally, a kernel trace on the `sched_wakeup` and `sched_switch` tracepoints [7] shares no probe or data with the application-level rate. It predicts that rate to within a third across 3 configurations, with nothing fitted: ratios 0.78, 1.06, 1.32, no consistent sign. The middle ratio comes from a configuration our own tracer check, fixed in advance, withholds; the two it admits still straddle one. The probe is not free: the rate was 0.272 untraced against 0.231 traced ($z = 3.6$), so each prediction is compared with the traced run's own rate.

The mechanism belongs neither to our Python clients nor to one processor family. A later registered campaign on three further pairs of machines found the same fall with the delivery in Kafka's own Java client, and the same removal by real-time priority (Supplement Part V). The Java client's rate was a third to a quarter of the Python client's. The same negative spans appeared on Neoverse N2 processors as on x86. Our timestamp is a CPython call and also waits on the interpreter lock; the Java client, which has none, shows the same failure at a lower rate.

C. Where the timescale comes from

The scheduling delay behind these negatives is not a single heavy tail. Over the 551,956 traced wakeups the counts have 3 local maxima. The last, in the 2–4 ms bucket, holds 10.5% of all wakeups and stands 4.5 $\times$ its lower neighbor. Above it the counts collapse: they fall 5.0 $\times$, 3.4 $\times$ and 4.7 $\times$ over the next three octaves and then 357 $\times$, where a power law falls by one factor every time (Fig. 3). Gregg reported the distribution as bimodal in the post introducing `runqlat` [7], and the `timerlat` tracer separates its interrupt and thread components [8]. What we add is the upper mode's position. It is consistent with the scheduler's base slice: the kernel's published rule [9] gives $4 \times 0.75 \text{ ms} = 3 \text{ ms}$ on the 8 cores the campaign's load recovers. That slice is derived rather than measured. With run-to-parity, a task's lag is held within one slice plus one tick [10].

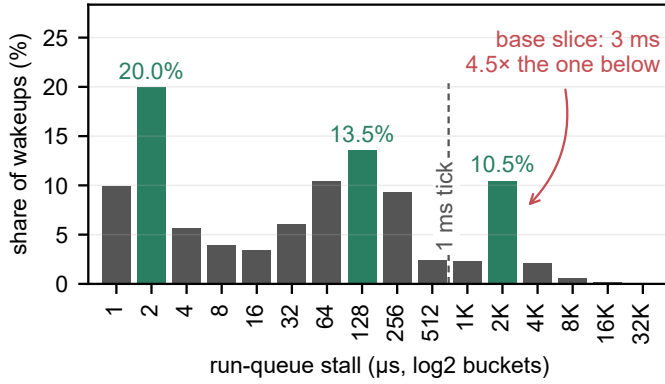


Fig. 3. **Traced run-queue stalls are trimodal.** 551,956 wakeups, bpftrace log₂ buckets, modes shaded. The jitter core peaks in the 2–4 μs bucket, which alone holds 20.0% of all wakeups; a second mode at 128–256 μs holds 13.5%; the third, at 10.5%, sits in the bucket holding the scheduler’s derived base slice of 3 ms. A power law of any index above one would have to descend monotonically across this axis. The count survives a 2× widening of the buckets in both phasings, so it is not an artifact of where this tool put its edges.

A thread woken while another task holds the processor waits out what is left of that task’s slice, up to a whole one. That is 1.02–4.29 times the per-condition deliveries this corpus measures, 0.70–2.95 ms. The registered campaign timed that wait afterwards, acknowledgment by acknowledgment, and found the rest of a slice rather than a whole one. That reading was made after its verdicts (Supplement S16.10). The slice lands on the scale of the interval being measured, which is why the check fires on a substantial minority of events. The scheduler’s tick is also 1 ms here, by coincidence of configuration; it is not the timestamp resolution τ .

D. What it costs, and the repair

The acknowledgment lag A is a scheduling quantity. It does not grow because a message travelled further, so the relative error of the transport proxy is A/D and falls as $1/D$. At our median acknowledgment lag, $A = 725 \mu\text{s}$, the error is 72% at a 1 ms delivery, 7% at 10 ms and 0.7% at 100 ms. Below 0.72 ms the lag exceeds the delivery it is a correction to. Across conditions A runs 500–1900 μs between its tenth and ninetieth percentiles. That puts the 10 ms error at 5–19% and the crossover at 0.50–1.90 ms (Fig. 4).

For a harness that reports S , two consequences follow on the gated corpus. The reported span is 24% of the delivery it stands for, understating it 4.2×. That is a median of per-condition ratios over 70 conditions, with an interquartile range of 4.0–7.5×. And because the two brokers’ clients timestamp differently, the proxy inflates the gap between two systems 1.7× over 24 matched workloads, a property of the clients that a reader would attribute to the brokers.

The acknowledgment lag can be added back. Equation 1 holds per event and A is producer-side on one clock. A harness that records its own acknowledgment lag therefore recovers the delivery it failed to measure. Its median error is 3.4% on the 42 conditions the check passes and 12.2% on the 28 it rejects, with upper quartiles of 20% and 19%: two populations the data cannot tell apart (Supplement S16.9). This does not reopen the

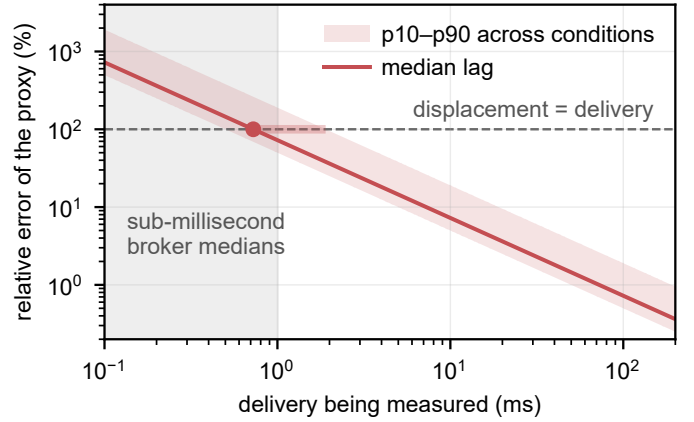


Fig. 4. **Where a path sits on the exposure curve.** The proxy’s error A/D against the delivery measured: the line at the median lag, the band over the central 80% of lags across conditions. On the dashed line, which the median crosses at 0.72 ms, the lag is as long as the path itself, and the shaded decade holds the sub-millisecond medians brokers now publish. A harness that times from the send subtracts no lag and has no curve.

check of Section II-D: the check declines to publish the proxy as a latency, and this reports the delivery instead.

IV. FAILURE 2: A MILLISECOND CLOCK AND A POSITIVITY FILTER

The cross-broker benchmark we audit computes its reported latency distribution from a subset of the samples it collects. It does not record how large that subset was, and it lets the send rate decide. Everything measured below is the OpenMessaging Benchmark in embedded mode, where driver and workers run in one JVM; its distributed mode failed on every attempt (Section VIII).

A. The filter, and its origin

In `LocalWorker.java:294` the end-to-end latency is now `-publishTimestamp`. Here now is read in the consumer and `publishTimestamp` is Kafka’s producer-set `CreateTime`, both in milliseconds. The difference is promoted to microseconds. The *positivity filter* at `WorkerStats.java:95` then admits a sample to the histogram only when `endToEndLatencyMicros > 0`. The message is counted as received a few lines earlier, but a non-positive latency is dropped and no counter records how many. Over the campaign below that silence covers 11,532,492 samples.

The filter’s origin is documented and its reasoning is sound. It was added in 2018 because end-to-end latency “is calculated based on time difference of `System.currentTimeMillis()` on two different machines” and “can be negative” [3]. The recorder is an `HdrHistogramRecorder`, which rejects negative values outright, so filtering for positives is the reasonable local fix, defensive rather than evasive. A benchmark that times a message faster than light has two choices: announce the discovery, or delete the message. This one deletes it. The fix is sound locally, and it has two non-local consequences. First,

> 0 also deletes every sample that computes to exactly zero, and with a millisecond clock every delivery shorter than a millisecond does. Second, the one observation that would show the measurement had failed is the exact observation the filter removes, and Section V finds the same uncounted disposal in four independent tools. The filter’s scope is telling. It sits on the end-to-end path, timestamped in milliseconds, while the same harness times its publish span in nanoseconds and records every sample. The coarse clock and the filter both landed on the one span that crosses processes.

B. What it did

Of the 75 instrumented cells whose own summary we captured, 71 report a median of 1.0 or 2.0 ms. Those medians were computed from between 0.36 and 100% of the samples the benchmark took (Fig. 5b). Two replicates of one cell, 200 B at 500 msg/s on one host, kept 431 and 120,434 samples, a 279-fold difference. Both reported a median of 1.0 ms, with nothing in the output to distinguish them. Three uninstrumented runs printed p50, p95, p99 and max all equal to 1.0. Like a stopped clock, the printed median is exactly right whenever the true one happens to be a millisecond. Over the full ledger of 223 instrumented runs the retained fraction falls as low as 0.0044%, about four samples in ninety thousand. The only cells that escape are the four whose payload is large enough that the resolution stops binding.

C. The retention law, and a registered confirmation

Publish latency across these cells takes only two values, 0.3 and 0.4 ms, while retention varies 279-fold. What moves retention is not the path but where the producer’s send instants fall against the *grid*: the instants, one millisecond apart, at which the timestamp changes value. A sample survives when its delivery crosses a grid instant, which for $T_{\text{true}} < \tau$ happens with probability T_{true}/τ (Fig. 5a):

$$\text{retention} = \min\left(1, \frac{T_{\text{true}}}{\tau}\right) \quad (\text{uniform phases}). \quad (4)$$

All seven configurations whose send interval is incommensurate with τ sit near 50%, with medians from 47.2 to 51.5. That implies $T_{\text{true}} = 0.49$ ms at $\tau = 1$ ms. Where the send interval is commensurate with the tick, the retained fraction is confined to a few values, so moving T_{true} moves a configuration between them (Supplement S11 tests every configuration). That prediction was registered before its runs, and it came true. Replicate spread collapses to 13.6 points at 32 KB and re-opens to 26.7 at 64 KB, a boundary crossed in both directions by one manipulated variable (Supplement S10). One registered detail failed: the 32 KB configuration pins at 69.2% retention, not on the predicted vertex at 66.67%.

The arithmetic is not ours. Hewlett-Packard’s Application Note 162-1 [11] carries the retention identity, the reduced fraction the grid turns on, the ceiling on resolution gain, the symptom and the cure (Supplement S17 sets them side by side). What is new is the sign of the operation. The counter keeps both readings, and the retained fraction is its estimator. A harness released in 2018 throws away the quantity a 1970 counter measured with.

D. Our own harness, and the sign channel

To show the deletion is not an artifact of one JVM, we reproduced the filter in an independent Python harness sharing no code with the benchmark. On one clock it recorded zero negative differences across 905,040 samples, as a causal chain requires, and the grid behavior reappeared intact. Separating the discards by sign is what makes the filter’s cost legible. Across the Kafka-driver corpus’s 214 runs, its 10,913,263 discarded samples contain not one negative: the most negative value at any load, size or rate is $0 \mu\text{s}$. Those discards are resolution failures, not clock failures, a distinction an unsigned counter cannot make. The same channel caught the real thing in the Redis-driver replication’s 9 runs: 41,403 negatives, every one exactly $-1000 \mu\text{s}$. That is the smallest value a one-millisecond timestamp can express, and it is no coincidence. The Redis driver’s span is a difference between two millisecond-floored clocks on two hosts. A sub-tick offset between two floored clocks can produce exactly one negative value, $-1000 \mu\text{s}$, and no other. Clock discipline was clean at every logged minute: this failure needs no misbehaving clock, only two of them and a floor (Supplement S14).

V. HOW WIDESPREAD

The disposal is a pattern across the tooling. We read ten tools at source, across five languages and nine vendors, and classified each with the code that classified the benchmark (Table III). Five dispose of the sample without counting it, in three classes: filtering, refusal by the tool’s own library, and substitution. Substitution is the worst because it leaves nothing missing: `fi` and `btt` substitute zero and one nanosecond, and a value never measured enters the distribution. Apache Pulsar’s performance client admits zero [12]. It drops negatives and keeps the samples the benchmark deletes, which then pile up visibly at zero. Apache Kafka’s own end-to-end tool keeps every sample and loses it anyway. It fills the array its percentiles come from by integer division, so on a sub-millisecond path every percentile it reports is zero [4].

In November 2025 Kafka’s coordinator metrics adopted the same disposal, clamping negative durations to zero with no counter [13]. Kafka’s record timestamp is itself a millisecond field on the wire, so patching one benchmark does not reach the broker’s own metadata (Supplement S16.7). The failure is not universal. Two tools pass negatives on unfiltered, as RFC 7679 asks [14], [15], and one counts its discards in a metric that names the cause [16]. What survives is the narrower claim: the tools that quantize or filter do so without counting.

Whether published results sit in this regime is a question we registered before reading any report (Supplement S25.1). The signature fixed in advance is an end-to-end median of exactly 1 ms, the smallest value the benchmark can print and the value every retained sample takes on a sub-millisecond path. Of 43 published reports that print the benchmark’s end-to-end percentiles, over 1,100 configurations, eight configurations in two reports carry it, both in public repositories, none in a paper or a vendor’s post. None of them also prints a p99 of exactly 1 ms, and no report states how many samples it kept. The

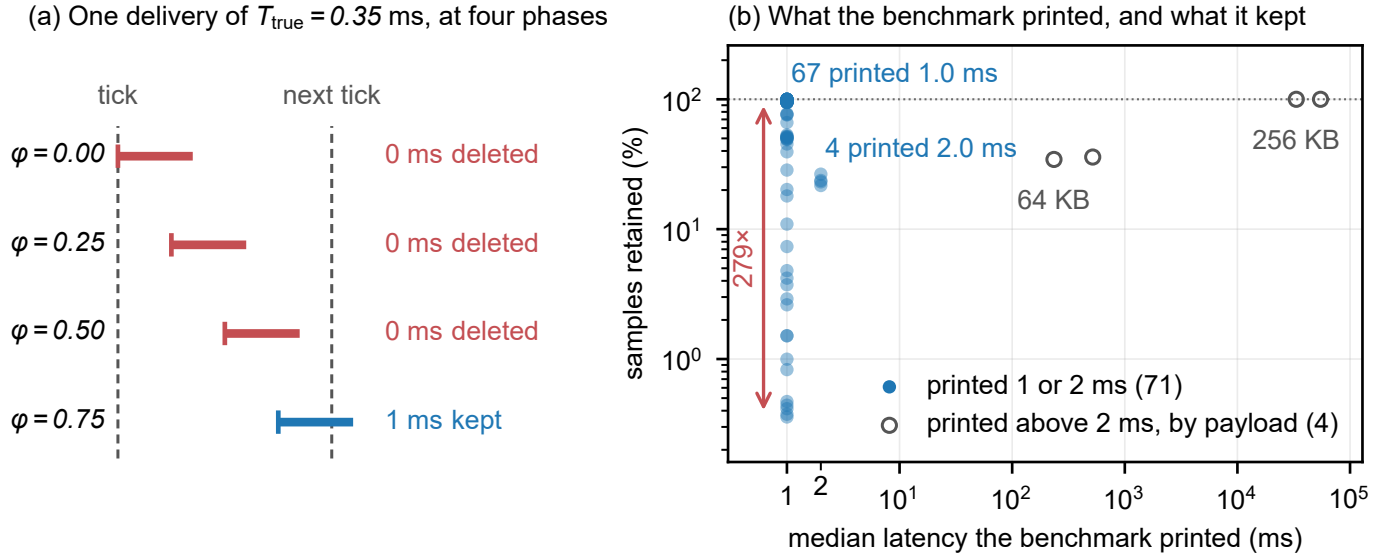


Fig. 5. **Why a sample is deleted, and what the deletion did.** (a) One delivery of 0.35 ms at four phases of one tick: three finish inside the tick, compute to 0 ms and are deleted; the fourth crosses the tick and is kept, as exactly 1 ms. (b) Every one of the 75 cells that printed a summary of its own, at the median the benchmark printed and the share of samples it kept. 71 cells print 1.0 or 2.0 ms while the retained fraction spans 279 $\times$; away from the grid the harness behaves.

TABLE III

TEN TOOLS, READ AT SOURCE. THE SPAN EACH MEASURES, THE CLOCK IT READS, WHAT HAPPENS TO A VALUE AT OR BELOW ZERO, AND WHETHER THE TOOL COUNTS WHAT IT DISCARDS; ONE ROW PER SPAN WHERE A TOOL MEASURES TWO. THE SOURCE LINE BEHIND EVERY ROW IS IN THE ARTIFACT'S REGISTRY, WHICH SUPPLEMENT S25 DESCRIBES.

Tool	Span measured	Clock	What happens to the value	Counts
Apache Kafka (bundled)	end-to-end	nanosecond	truncate to the resolution	no
Apache Kafka (bundled)	publish	millisecond	truncate to the resolution	no
Apache Pulsar perf	end-to-end	millisecond	filter, ≥ 0	no
NATS CLI	end-to-end	nanosecond	—	no
OpenMessaging Bench.	end-to-end	millisecond	filter, > 0 , truncate to the resolution	no
OpenMessaging Bench.	publish	nanosecond	—	no
RabbitMQ PerfTest	end-to-end	nanosecond	—	no
Rezulus	scheduler wakeup	nanosecond	—	yes
blktrace/btt	block stage	nanosecond	substitute a constant	no
emqtt-bench	end-to-end	millisecond	—	no
fio	I/O completion	nanosecond	substitute a constant	no
wrk2	end-to-end	microsecond	refused by own library	no

regime is reached in published practice, but rarely. One reason is visible in the same reports: 14 of them print medians below a millisecond, which the upstream clock cannot produce, from copies of the benchmark whose maintainers changed how the span is timed.

VI. WHAT BENCHMARK AUTHORS SHOULD DO

Both failures are cheap to detect, and Table IV lists the checks. SPEC and TPC judge a benchmark on five criteria [17, Sec. 3]. The tool we audit passes the half of *reproducibility* a reader checks, run-to-run consistency at a fixed configuration, because it fails *verifiability*. A median from 0.36% of the samples is the more stable for having its variation deleted, and a second tester who picks a different send rate measures a different sample. Better measurement systems exist [18], [19], [20], and the checks apply to whatever they measure. Every check in the table restores verifiability, and every one has a precedent. Network measurement has published the share of traces it rejected for nearly thirty years [21], [22].

OWAMP records a packet timestamped in the future unaltered rather than discarding it [23]. Randomizing probe times against periodic phenomena is standing advice [24], [25]. What we add is the conjunction, applied where a benchmark reports a latency, with the rejection rate published as a headline quantity.

The last check is the one mitigation our measurements support directly: real-time priority on the timestamping threads cut the negative-span rate 7–80 $\times$ at unchanged background load. Busy-polling a dedicated core should buy the same for the price of that core, though we did not measure it. The floor is not zero, so the check stays, and a real-time or polling timestamping thread changes the system it measures; where that is unacceptable, report retention instead.

A better-synchronized clock fixes neither failure. Hardware-timestamped PTP [26] would take two hosts from our 67 μ s of *chrony* agreement to under a microsecond. Behind a one-millisecond timestamp the two produce byte-identical records: the filter deletes the same samples and the retention arithmetic is unchanged. A finer clock does fix the second failure on these

TABLE IV
CHECKS THAT COST NOTHING, AND WHAT EACH CATCHES.

Check	Catches
Sign-check every run under the load it is quoted at, and publish the rejection rate.	Failure 1: an origin timestamp that failed (Section III-A).
Time from the send; or record the acknowledgment lag and add it back.	Failure 1's error, A/D (Section III-D).
Publish the retained fraction and the discards counted by sign beside every latency.	Failure 2, and which clock failed (Section IV-D).
Read your own percentiles: all on the clock's grid means the grid made the distribution.	Failure 2, from output already printed (Section IV-B).
Dither the send instant and record the achieved rate.	Retention fixed by the schedule's phase (Section IV-C).
Make the timestamping thread unpreemptable, and say that you did.	Failure 1 at its source (Section III-B).

paths. It retires Equation 4's deletions and moves the grid to faster paths, but it leaves the filter and every genuine negative in place. The remedies are therefore ordered: record at native resolution, count what is discarded, dither the send instant, and only then does synchronization quality matter.

Once the failed runs are removed, the broker comparison that motivated the work is small. On the end-to-end latency, which is a causal chain, the two brokers are equivalent against a wider margin (Supplement S13); on the transport proxy they sit within a millisecond (TOST against a 1 ms margin, $p < 0.001$, across 3 levels). So it is not a purchasing argument. One consumption pattern, which a co-located testbed hides, reverses their order. Our own measurement is not symmetric across that comparison. Kafka's client deserializes the payload before t_{recv} and Redis's after it, which is worth $19.5 \mu\text{s}$ or 2.4% of Redis's median delivery. No reader could have known that had we not said so, and the end-to-end latency is free of it.

VII. RELATED WORK

That measurement error can dominate a benchmark is established. What prior work does not supply is a check applied to every run, and a count of what the check removes.

A. Broker benchmarks

Broker comparisons appear continuously in peer-reviewed venues and in the gray literature. Apache Kafka and Redis Streams are the pair they most often name, and the OpenMessaging Benchmark [2] is the tool they share. That experimental setup can dominate a result is established [27], and how to report one is a settled literature [28], [29]. What that literature does not do is check what its tool discards. Karimov et al. [30] separate event-time from processing-time latency. Van Dongen and Van den Poel [31] use single-broker append timestamps "to ensure correct latency measurements": our one-clock rule, reached earlier and by another route. Neither asks what a millisecond timestamp does to a sub-millisecond path, nor which samples the tool keeps. Where a comparison reports a median above the tool's one-millisecond timestamp resolution, the deletion arithmetic of Section IV does not

bind. Not every comparison does: a 2026 comparison names a percentile for one broker and gives the other two only "sub-millisecond" [32], so its ranking cannot be placed against the resolution at all.

B. Cross-process time

Lamport [33] established that a distributed system has no single physical time, only a causal partial order, and that order decides this paper's central span. Measurement systems defeat clock disagreement [26] by calibrating cross-node counters [18], timestamping in the network interface [19] or the switch dataplane [20], and measuring the timer's own accuracy [5]. Villain et al. [34], profiling FreeBSD against a hardware reference, found that the outgoing socket timestamp "does not respect causality" under every stress pattern, including none. They attributed host latency to interrupt processing, scheduling and context switching: the mechanism of Section III, measured more precisely than we measure it. What it does not give is the consequence for a two-process measurement: the negative-span rate falls as the delivery being measured grows. The manipulations that establish it separate scheduling from its rival explanations (Section III-B). Paxson [21] treated physically inconsistent transit times as measurement failure and published the share of traces affected. Lancet [19, Sec. 4.3] reports an inconclusive verdict when a run's samples fail its stability test. Neither reports the share of runs a campaign lost. Production tracing reached a related answer without a mechanism: skew adjusters that rewrite a span's timestamps are called harmful and have defaulted to off since 2020 [35]. Sharma et al. [36] count negative timing spans in distributed inference systems. They hold that "queueing alone cannot produce negative timing spans" and read the negatives as clock skew. For the queue they model, we agree. A second queue decides ours: the run queue the timestamping thread waits in.

C. Resolution and discarded samples

That a timer coarser than the interval it measures invalidates the measurement is textbook [5]. Metrology carries both failures in one uncertainty budget: the NIST stopwatch guide budgets reaction time beside display resolution, and notes that at short intervals the resolution becomes significant against the stopwatch's rated accuracy [37]. That periodic sampling commensurate with a periodic phenomenon sees only part of it is the IPPM framework [24], and RFC 3432 makes a randomized start mandatory for periodic streams [25]. Coordinated omission [38] is the mirror image. There the measurement fails to record samples that would have been slow; here it deletes samples that were fast. Millsap and Holt argue that coarse timing survives in aggregate because the two signed errors cancel [39, Sec. 7.7]. A filter admitting only positive samples removes one sign, and with it the cancellation that defense depends on.

a) *Sources of scheduling delay*: A runnable thread does not always run. Work-conservation violations are documented

for CFS-era schedulers [40], ours being EEVDF-era. Chandrasekar and Kramberger [41] apply a single-parameter queueing law to benchmark bias and reach our inflation half independently. The load-geometry contrast of Section III-B refutes that single-parameter form, not queueing itself.

VIII. THREATS AND LIMITATIONS

Attributing the negatives to scheduling is an inference, separated from its rivals by manipulation. One clock excludes clock skew, and identical ρ at rates twofold apart excludes utilization. `SCHED_FIFO` excludes timer noise, which does not respect scheduling class. A longer transport that lowers the rate excludes stress. A tracepoint histogram that predicts the rate unfitted is direct observation. The interpreter lock is bounded twice: the kernel-only estimator, which cannot see the lock, already brackets the observed rate, and Kafka's Java client shows the same failure (Section III-B). Clock incoherence across virtual CPUs [42] is closed twice as well. The probe's 90 ns increment admits only `kvm-clock` or `tsc`, both of which the kernel uses solely where cross-CPU coherence is asserted, and the send-referenced span's floor is $455\times$ too high to hide the deepest negative (Supplement S15). Across hosts `chrony` bounds its own error at 4–7 ms per host, so we claim no cross-host bound in general, and the skew threshold of Section VII-B does not transfer to a loaded machine (Supplement S33).

The mechanism requires a timestamping path that can be preempted. Designs that pin a thread to a dedicated core and busy-poll, or that timestamp in hardware, remove the preempted state, and we make no claim about them. The mechanism's constants are those of one EEVDF-era kernel (6.8), and CFS-era schedulers may shift them. A registered campaign tested the mechanism's law on three further pairs of machines. One prediction held on every campaign: load raises the negative-span rate. On Kafka real-time priority removed the negatives, and the number of online CPUs moved the fall through the slice it sets. Its predictions of where the fall sits and how wide it is failed, and Python's rate exceeded Java's by more than the registered factor of 1.5 (Supplement Part V). The deletion arithmetic is demonstrated on one benchmark and reproduced in one independent harness, and the registered audit of Section V found its signature in few published results, so we claim the regime for those and for our own runs, not for the literature at large. The ten tools of Section V were read at source, not deployed. The benchmark's distributed mode failed on every attempt, so its cross-host exposure rests on source reading and our own two-host harness (Supplement S24). Exposure depends on a deployment's path latency and producer rate, which is why we recommend publishing a retention rate rather than calling any published number wrong.

IX. CONCLUSION

A latency benchmark measures the system only while its timestamps are taken faster than the delivery it measures. Once the message is the faster of the two, the arithmetic reports it faster than light, and the benchmark deletes it. Two

timestamps written by threads that each wait for a core invert the delivery between them whenever those waits differ by more than it. A timestamp quantized to a millisecond and filtered for positivity deletes most of a sub-millisecond path, by arithmetic the operator never sees, and reports the survivors as exactly one millisecond. Neither failure announces itself. A finer clock cures only the second, and neither is expensive to detect. Check the spans whose endpoints are timestamped in different places, count what your tool throws away, and publish the retention rate beside the number.

ACKNOWLEDGMENT

The authors thank N. Herbst, for the load-generation point that became one of the checks above, for pointing us to the timer study cited in Section VII, and for a reading that reset the abstract.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, manuscript editing, and literature search. Experimental design, scientific claims, and interpretation are the authors'.

ARTIFACT AVAILABILITY

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, `v3.0.0`, whose files are restricted pending publication. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

REFERENCES

- [1] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," <https://github.com/openmessaging/benchmark/issues/216>, 2021.
- [2] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.
- [3] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit `ebb5d6b8`, Mar. 2018, <https://github.com/openmessaging/benchmark/pull/56>. Accessed: Aug. 18, 2026.
- [4] Apache Software Foundation, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, <https://github.com/apache/kafka>. Accessed: Aug. 31, 2026.
- [5] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.
- [6] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: <https://github.com/statsbomb/open-data>
- [7] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," <https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html>, Oct. 2016, accessed: Sep. 13, 2026.
- [8] D. Bristol de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, 2025.
- [9] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit `2ae891b82695`, 2025.
- [10] V. Guittot, "sched/fair: Manage lag and run to parity with different slices," Linux kernel mailing list, Jul. 2025.
- [11] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766; undated, year from its *HP Journal* citation.
- [12] Apache Pulsar contributors, "PerformanceConsumer.java," Source repository, 2026, <https://github.com/apache/pulsar>. Accessed: Aug. 21, 2026.

- [13] Apache Kafka contributors, “KAFKA-19888: Clamp negative values in coordinator histograms,” Issue tracker and pull request #20986, 2025, <https://issues.apache.org/jira/browse/KAFKA-19888>. Accessed: Aug. 21, 2026.
- [14] Broadcom, “RabbitMQ PerfTest — Consumer.java,” Source repository, 2026, <https://github.com/rabbitmq/rabbitmq-perf-test>. Accessed: Aug. 31, 2026.
- [15] Synadia, “NATS cli — cli/latency_command.go,” Source repository, 2026, <https://github.com/nats-io/natscli>. Accessed: Aug. 31, 2026.
- [16] IOP Systems, “Rezulus: systems performance telemetry — scheduler_discarded_samples,” Source repository, 2026, <https://github.com/iopsystems/rezulus>. Accessed: Aug. 20, 2026.
- [17] J. von Kistowski, J. A. Arnold, K. Huppler, K.-D. Lange, J. L. Henning, and P. Cao, “How to build a benchmark,” in *Proc. 6th ACM/SPEC Int. Conf. on Performance Engineering (ICPE)*, 2015, pp. 333–336.
- [18] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, “Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads,” *arXiv preprint arXiv:2205.09325*, 2022.
- [19] M. Kogias, S. Mallon, and E. Bugnion, “Lancet: A self-correcting latency measuring tool,” in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896.
- [20] R. Kundel, “P4STA: High precision network function benchmarking,” in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.
- [21] V. Paxson, “On calibrating measurements of packet transit times,” in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.
- [22] —, “Strategies for sound Internet measurement,” in *Proc. ACM Internet Measurement Conf. (IMC)*. ACM, 2004, pp. 263–271.
- [23] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, “A one-way active measurement protocol (OWAMP),” Internet Engineering Task Force, RFC 4656, Sep. 2006.
- [24] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, “Framework for IP performance metrics,” Internet Engineering Task Force, RFC 2330, May 1998.
- [25] V. Raisanen, G. Grotefeld, and A. Morton, “Network performance measurement with periodic streams,” Internet Engineering Task Force, RFC 3432, Nov. 2002.
- [26] IEEE, “IEEE standard for a precision clock synchronization protocol for networked measurement and control systems,” IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.
- [27] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.
- [28] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.
- [29] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.
- [30] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, “Benchmarking distributed stream data processing systems,” in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, 2018, pp. 1507–1518.
- [31] G. Van Dongen and D. Van den Poel, “Evaluation of stream processing frameworks,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.
- [32] Index.dev, “NATS vs Redis vs Kafka: message broker comparison,” Practitioner comparison, 2026, <https://www.index.dev/skill-vs-skill/nats-vs-redis-vs-kafka>. Accessed: Sep. 3, 2026.
- [33] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [34] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, “Probing the latencies of software timestamping,” in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6.
- [35] Jaeger contributors, “Clock skew adjuster considered harmful (issue #1459),” Issue tracker, 2019, <https://github.com/jaegertracing/jaeger/issues/1459>. Accessed: Aug. 21, 2026.
- [36] A. Sharma, D. Shah, D. Larivière, and H. ElBakoury, “Time, causality, and observability failures in distributed AI inference systems,” *arXiv preprint arXiv:2604.21361*, Apr. 2026, Open Compute Project, Unified Intelligent Infrastructure workstream.
- [37] J. C. Gust, R. M. Graham, and M. A. Lombardi, “Stopwatch and timer calibrations (2009 edition),” National Institute of Standards and Technology, NIST Special Publication 960-12, 2009.
- [38] G. Tene, “How NOT to measure latency,” 2015, talk, Strange Loop / QCon.
- [39] C. Millsap and J. Holt, *Optimizing Oracle Performance*. Sebastopol, CA: O’Reilly, 2003.
- [40] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, “The Linux scheduler: a decade of wasted cores,” in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.
- [41] A. Chandrasekar and J. Kramberger, “Identifying and mitigating systemic measurement bias in production LLM inference benchmarks,” *arXiv preprint arXiv:2605.24217*, 2026.
- [42] Linux Kernel Documentation, “KVM-specific MSRs: MSR_KVM_SYSTEM_TIME_NEW,” <https://docs.kernel.org/virt/kvm/x86/msr.html>, 2026, accessed: Aug. 21, 2026.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the Universität Stuttgart, Germany, and the Université de Lorraine, France. He holds an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.

Romarie Duvignau received the Ph.D. degree in computer science from the University of Bordeaux, France, in 2015, for work on the maintenance and simulation of dynamic random graphs. He is an Associate Professor and Head of the Network and Systems unit at Chalmers University of Technology, Gothenburg, Sweden. His research concerns distributed and data-driven algorithms, the monitoring of large-scale distributed systems, and stream analytics for cyber-physical systems.